Ansible Automation Platform 2.7

Automation Portal and Content Lifecycle

Version 1.0.0, 2026-08-27

Home

Introduction

Course Title: Ansible Automation Platform 2.7: Automation Portal and Content Lifecycle

Description: Course description FIXME

Duration: FIXME hours

Topics covered in this course

- Automation Portal
- Visual Execution Environment Builder
- Centralized Content Discovery

Prerequisites

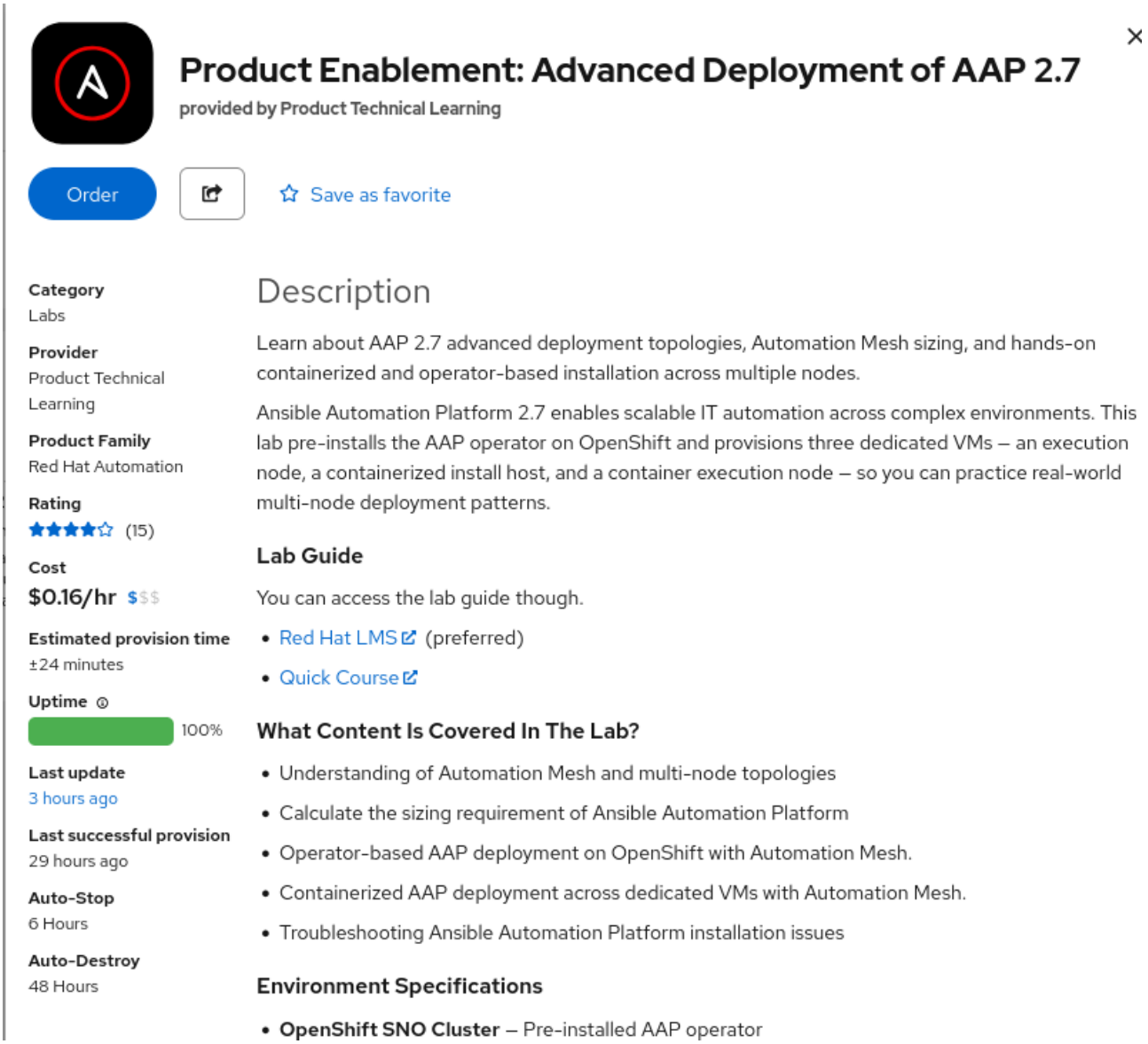
This course assumes that you have the following prior experience:

- Course or experience…
- Course or experience…
- Course or experience…

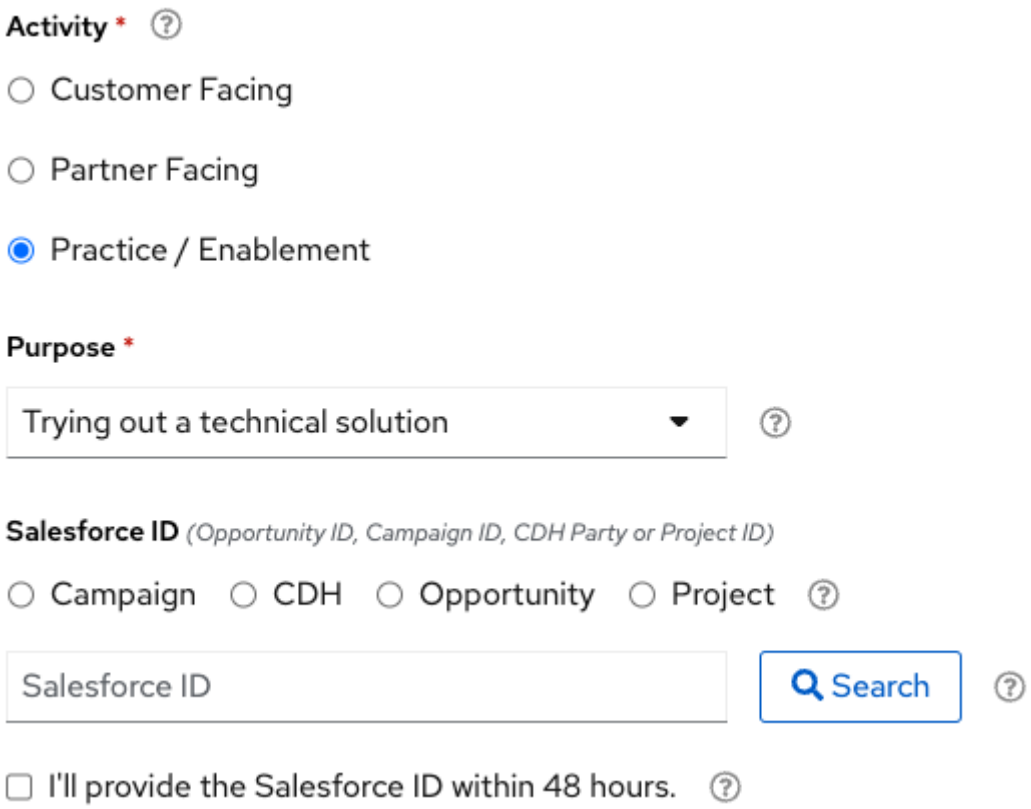
Lab Environment

Instructions to Launch Your Lab on the Red Hat Demo Platform (RHDP)

1. **Log in** to the RHDP portal.
2. **Search** for the catalog entry: **Advanced Deployment of Ansible Automation Platform 2.7**.



3. **Click** on the catalog entry in the search results.
4. On the catalog page, **click** the **Order** button.
5. **Fill out** the required details in the order form.



6. **Review the warning** at the bottom of the form and **check the box** labeled: **"I confirm that I understand the above warnings."**
7. **Click** the **Order** button to place your lab order.

Important Notes:

- This lab may take approximately **20 minutes** to become ready.
- You will receive an **email with access details** once your lab environment is ready.
- You can also **retrieve lab access** directly from the RHDP portal.

How to Access Your Lab via the RHDP Portal:

1. On the RHDP portal, **click** on the **Services** option in the left-hand menu.
2. **Select your lab** from the listings on the right-hand side of the page to view access details.

RHDP Portal Links

- RedHat associates: https://demo.redhat.com/
- RedHat partners: https://partner.demo.redhat.com/

Automation Portal

The Ansible Automation Portal in AAP 2.7 serves as a unified interface that shifts the platform focus from simple execution to a comprehensive "build, discover, and run" experience. It simplifies enterprise automation for users of all skill levels by replacing complex CLI workflows with a streamlined, point-and-click web experience.

Key features include:

- **Simplified User Experience:** Designed for business users, reducing the need for deep technical knowledge of Ansible playbooks.
- **Platform Integration:** Deployed on OpenShift Container Platform (OCP) with support for RBAC-based navigation, allowing administrators to reduce UI complexity through role-based access policies.
- **Role-Based Access Control:** Facilitates user role separation, ensuring that users only interact with relevant automation assets.
- **Unified Workflow:** Acts as a centralized hub for managing the full lifecycle of automation, from building Execution Environments to discovering and running pre-defined automation content.

Automation Portal: Installation Overview with Architectural Benefits

The Automation Portal with Ansible Automation Platform provides a centralized interface for managing automation workflows, visual environment building, and content discovery. When deploying on OpenShift Container Platform (OCP), the Automation Portal leverages the Kubernetes-native architecture to ensure scalability, high availability, and seamless integration with existing AAP controller components.

The Evolution of the Portal

In previous iterations, such as AAP 2.6, the portal functioned primarily as an execution layer. Users navigated to the interface, identified authorized job templates, populated surveys, and triggered automation.

Automation Portal 2.7 expands this scope to support three primary pillars: * **Visual Execution Environment (EE) Builder:** A GUI-driven workflow for container image creation. * **Centralized Content Discovery:** A unified catalog that aggregates automation assets from multiple sources. * **Platform-wide Governance:** Enhanced RBAC controls, administrative navigation policies, and AI-assisted operational workflows.

Architectural Benefits

The shift toward this "Build, Discover, and Run" architecture provides several key advantages for organizations:

1. Unified Developer Productivity

By integrating the EE Builder and the Content Discovery catalog directly into the portal, developers no longer need to switch context between the command line, CI/CD pipelines, and the execution interface. This reduces the "friction of automation," allowing teams to define, build, and deploy automation assets without leaving the AAP ecosystem.

2. Reduced Complexity via Admin-Defined RBAC

The architectural design of the 2.7 portal emphasizes role-based navigation. Administrators can configure custom menus to match user roles: * **End-Users:** May see a simplified view restricted to "Templates" and "Job History," minimizing cognitive load and accidental configuration changes. * **Platform Engineers:** Retain access to the full portal, including builder tools, catalog management, and administrative policy settings.

3. Streamlined Governance and Discovery

The architecture supports multi-source aggregation. By centralizing content from various repositories (GitHub, GitLab, Private Automation Hub), the portal provides a "browse-before-build" workflow. This allows teams to trace existing automation assets, preventing the duplication of effort and ensuring that teams are using approved, organization-wide standards.

4. Extensibility and AI Readiness

With the inclusion of an MCP (Model Context Protocol) server, the portal is architected to support AI-assisted operations. This allows for natural language interaction with the platform, enabling users to query automation status or trigger workflows through supported AI clients, further lowering the barrier to entry for non-technical stakeholders.

By consolidating these functions, Automation portal reduces the operational overhead associated with maintaining disparate build tools and content repositories, ensuring that automation scaling remains consistent with enterprise security and governance policies.

Prerequisites and Requirements

Before initiating the deployment, ensure your environment meets the following technical requirements:

- **Platform Subscription:** Valid Red Hat Ansible Automation Platform subscription.
- **Existing AAP Infrastructure:** An active instance of AAP 2.6+ to facilitate authentication and integration.
- **OpenShift Environment:** Access to an OpenShift Container Platform cluster (version 4.12–4.17+).
- **Permissions:** Cluster-level permissions to create OAuth applications within the target project.
- **CLI Tools:**
- `oc` CLI tool for OpenShift management.
- `helm` (version 3.10 or newer) for package deployment.

Installation Overview

The official deployment method uses **Helm charts** on OpenShift Container Platform. The Helm chart automates the deployment of the Automation Portal, handling networking, storage, and integration with your AAP instance.

Two-Phase Installation Process:

1. **Phase 1 - Prerequisites:** Complete Hands-on Setup: Deploying Automation Portal - Prerequisites and Configuration to set up OAuth, API tokens, and OpenShift secrets.
2. **Phase 2 - Deployment:** Deploy the Helm chart using Hands-on Lab: Installing and Verifying the Automation Portal on OCP.

Both phases are required for a successful installation.

Managing RBAC-based Navigation Menus for User Role Separation

In modern enterprise automation, "platform sprawl" is a significant hurdle. When users are presented with a cluttered interface containing tools they don't need—such as Execution Environment (EE) builders or Git repository management—it increases cognitive load and introduces potential security risks.

The Automation Portal addresses this by providing **RBAC-based navigation menus**, allowing platform architects to tailor the UI experience based on the user's specific role.

Related Pages: This page focuses on controlling which menu items are visible. For more information on API-level permissions and access control, see Reducing UI Complexity through Admin-Defined Access Policies.

Architectural Concept

By tying navigation visibility to RBAC (Role-Based Access Control) groups, administrators can enforce a "least privilege" visual experience. This ensures that: * **End Users:** Interact only with the automation services they need (Job Templates and History), reducing complexity. * **Platform Engineers:** Retain full visibility and administrative control over the underlying infrastructure, including EE definitions, content discovery, and synchronization settings.

Hands-On Lab: Configuring Menu Visibility

This lab demonstrates how to restrict the UI for standard operational teams while granting full access to the Platform Engineering group.

Exercise 4.1: Review Current Navigation Visibility

Before applying constraints, observe the default behavior of the portal.

1. Log in to the Automation Portal as a user with `admin` privileges.
2. Observe the sidebar; verify that all sections (Templates, History, EE Definitions, Collections, Git Repositories, Administration) are visible.
3. Log out and log in as a standard user (e.g., `clouduser1`).
4. **Verification:** Note that by default, standard users may see the entire menu structure, even if they lack the underlying permissions to perform tasks within those sections.

Exercise 4.2: Configure Navigation Menu RBAC

In this step, we map specific navigation items to user groups to clean up the UI.

1. **Log in** to the portal as an `admin`.
2. Navigate to **Administration** > **RBAC** > **Navigation Visibility**.
3. **Configure End-User Access:**
   - Select the groups: `cloud-team`, `network-team`, and `rhel-team`.
   - Enable visibility for the following items only:
   - `[x]` Templates
   - `[x]` History
   - Ensure all other items (EE Definitions, Collections, Git Repositories, Synchronization) remain unchecked.
4. **Configure Platform Engineering Access:**
   - Select the `platform-team` group.
   - Ensure all navigation items are checked, providing full access to administrative functions.
5. **Save** the changes.

Exercise 4.3: Verification

1. Log out as `admin`.
2. Log in as a user who is a member of the `cloud-team`.
3. **Result:** The sidebar should now dynamically hide the advanced development and administration features, displaying only **Templates** and **History**.
4. Log out and log in as a member of the `platform-team` to verify that all administrative menus are fully accessible.

Troubleshooting Tips

- **Cache Persistence:** If navigation items remain visible after updating permissions, ensure you

refresh your browser cache or initiate a hard reload.

- **Group Membership:** If a user cannot see the expected menus, verify they are correctly assigned to the intended group in the Identity Provider (IdP) or the local portal user database.
- **Overlapping Roles:** If a user belongs to multiple groups (e.g., they are a member of both `cloud-team` and `platform-team`), the system will aggregate the permissions, granting them the union of all visible menus. Ensure user assignments are strictly managed.

Reducing UI Complexity through Admin-Defined Access Policies

In large-scale enterprise environments, the Automation Portal can become cluttered with administrative interfaces, repository configurations, and low-level build tools. To improve user experience and maintain security compliance, the Automation Portal allows administrators to define access policies that dynamically tailor the UI based on user roles.

Related Pages: This page focuses on API-level permissions and access control. For information on controlling which menu items are visible to users, see Managing RBAC-based Navigation Menus for User Role Separation.

The Concept of UI Minimalism through RBAC

By leveraging Role-Based Access Control (RBAC), you can decouple the complexity of the platform from the needs of the consumer. Reducing UI complexity serves two primary goals:

- **Operational Efficiency:** End users are not overwhelmed by administrative overhead, reducing the cognitive load required to execute automation.
- **Administrative Governance:** By hiding configuration menus (like EE Definitions or Git Repositories), you prevent unauthorized changes to the underlying platform infrastructure.

Architectural Implementation

The portal maps navigation visibility directly to the user's existing Ansible Automation Platform (AAP) permissions. When a user authenticates, the portal checks their associated permissions:

1. **Identity Mapping:** The portal retrieves the user's roles via their API token.
2. **Policy Evaluation:** The system evaluates the user's role against the defined navigation access policies.
3. **UI Projection:** The frontend dynamically renders only the menu items for which the user has authorized access.

The UI policy does not replace server-side security. The system architecture ensures that even if a UI element were exposed, the AI agent and backend services will block any action not permitted by the server-side configuration.

Scenario: Role-Based Navigation Configuration

To implement the requested separation of duties, follow this logical configuration structure:

User Group	Visible Menu Items	Hidden/Restricted Items
End Users	Templates, History	EE Definitions, Collections, Git Repositories, Sync
Platform Engineers	Full Portal (All Items)	None

Configuring the Navigation Policy

To restrict navigation, administrators define access policies within the Portal settings:

1. **Define the Scope:** Identify the target Role (e.g., `cloud-team`) in the AAP RBAC system.
2. **Access Filtering:** Within the Automation Portal administrative settings, navigate to **UI Access Policies**.
3. **Map Groups to Menus:**
   - Create a policy for the `cloud-team`.
   - Set the `Allowed Navigation Items` list to strictly include: `['Templates', 'History']`.
   - Verify that `System Configuration` and `Builder` menus are excluded.

Practical Exercise: Validating Access Policies

This activity demonstrates how to verify that the portal successfully filters UI components based on the assigned role.

Prerequisites

- Ensure you have two distinct test accounts: `user-standard` and `user-admin`.
- Ensure `user-standard` is assigned to a restricted role in your IdP or AAP.

Step-by-Step Verification

1. **Login as Platform Engineer:** Log in with the `user-admin` account. Verify that the sidebar contains the full suite of options, including **EE Definitions** and **Git Repositories**.
2. **Apply Administrative Filtering:** Navigate to the Portal Admin panel. Select **UI Access Policies** and create a rule for the group containing `user-standard`. Select only **Templates** and **History**.
3. **Login as End User:** Log in with the `user-standard` account. Observe that the navigation sidebar has shrunk, removing all administrative configuration tools.
4. **Verify Backend Protection (Optional):** Attempt to access the URL for the EE Builder directly (e.g., `/portal/ee-builder`). The system should return a 403 Forbidden error, confirming that the UI policy is backed by robust server-side security.

Troubleshooting

- **Cache Issues:** If menu changes do not reflect, clear your browser cache or force-refresh the session.
- **Role Propagation:** If a user still sees extra menus, verify that their user role in AAP was updated and their current session token has been refreshed.

Hands-on Setup: Deploying Automation Portal - Prerequisites and Configuration

In this lab, you'll walk through the essential prerequisite steps to deploy the Ansible Automation Portal on OpenShift Container Platform. We'll configure OAuth authentication with AAP, generate required API tokens, and prepare your OpenShift environment for the portal deployment. These steps ensure secure communication between the portal and your AAP instance.

Overview of What We'll Do

By the end of this lab, you will have:

- Created an OAuth application in AAP for portal authentication
- Generated and secured API credentials
- Authenticated with your OpenShift cluster
- Created a dedicated project for the automation portal
- Prepared all necessary credentials for the Helm chart deployment

This is the foundational work that must be completed **before** you deploy the Helm chart. Think of it as setting up the "communication channels" between the portal and your AAP instance.

Task 1: Create an OAuth Application in Ansible Automation Platform

The OAuth application is how the Automation Portal will authenticate users against your AAP instance. Think of it as registering the portal as a "trusted application" within AAP.

Step 1.1: Log in to AAP

Open your Ansible Automation Platform instance in a web browser and log in as an administrator.

```
# Example AAP URL (replace with your instance)
https://aap.example.com
```

Step 1.2: Navigate to OAuth Applications

Once logged in:

1. In the left sidebar, click **Access Management**

2. Select **OAuth Applications** from the submenu
3. Click the **Create OAuth Application** button in the upper right corner

Step 1.3: Configure the OAuth Application

Fill in the following fields with these exact values:

- **Name:** `automation-portal`
- **Organization:** Select the organization where your automation content lives (typically "Default" unless you have a multi-tenant setup)
- **Authorization grant type:** Select **Authorization code**
- **Client type:** Select **Confidential**
- **Redirect URIs:** Enter a placeholder URL for now: `https://example.com/api/auth/rhaap/handler/frame` (we'll update this later with the actual portal deployment URL)

The Redirect URI is temporarily set to a placeholder because the actual portal URL won't exist until after you deploy the Helm chart. We'll come back and update this with the real deployment URL in a later step.

Step 1.4: Save and Capture Credentials

Click the **Create OAuth Application** button.

Once created, the system will display: * **Client ID** — copy this value and save it in a secure location (notepad, password manager, etc.) * **Client Secret** — this appears only once; copy and save it immediately

IMPORTANT: The Client Secret will never be displayed again. If you lose it, you'll need to regenerate it. Save both values now before navigating away.

Task 2: Enable OAuth Token Creation for External Users

1. In the AAP left sidebar, click **Settings**
2. Select **Platform gateway** from the submenu
3. Look for the setting: **Allow external users to create OAuth2 tokens**

If the setting is not already enabled:

1. Click **Edit platform gateway settings**
2. Toggle **Allow external users to create OAuth2 tokens** to **Enabled**
3. Click **Save platform gateway settings**

Verify that the setting now shows as **Enabled** on the Platform gateway settings page.

This setting controls whether users logging in via external identity providers (LDAP, SAML, OAuth, etc.) can create OAuth2 tokens. The portal uses this capability

> for seamless authentication.

Task 3: Generate an API Token for the Automation Portal

The API token is used by the Automation Portal to communicate with your AAP instance on a service-to-service basis. This token is what allows the portal to synchronize job templates, manage execution environments, and perform automation tasks.

Configure the Token

Fill in the following fields:

- **Description:** `automation-portal-api-token` (descriptive name for your reference)
- **Application:** Select the **automation-portal** OAuth application you created in Task 1
- **Scope:** Select **read** (the portal needs write access to manage templates and execution environments)

Generate and Save the Token and as the system will display the generated token — copy it immediately and save it in a secure location alongside your Client ID and Client Secret from Task 1.

> **IMPORTANT:** Like the Client Secret, the API token is displayed only once. If you navigate away, you cannot retrieve it again without creating a new token.

Make a note of the three values you now have:

- Client ID: _
- Client Secret: _
- API Token: _

These three values are critical for the next phase of setup.

Task 4: Authenticate with Your OpenShift Cluster

Now we shift focus to the OpenShift side of the setup. We need to connect to your OCP cluster using the CLI tool, which will be the gateway for creating the portal's project and secrets.

1. Login to the bastion host using the credentials from the RHDP login page.
2. Once login to bastion host then login to OCP cli using the command shared in **OpenShift CLI Commands** section on the RHDP login page :

```
oc login -u admin -p <openshift-admin-password>  https://api.cluster-
kstzt.dyn.redhatworkshops.io:6443
```

1. Run the following command, replacing `portal` with your preferred project name if you want a different project name(keep it lowercase with hyphens only, no underscores):

```
oc new-project portal
```

1. Make sure your CLI context is set to the newly created project:

```
oc project portal
```

1. Build and Deploy the Plugin Registry
2. Go to the Installer link and Get the installer link for **Ansible automation portal Setup Bundle**.
3. Download the installer file to the VM:

```
wget -O automation.tar.gz "<paste-link-here taken from step 5>"
```

4. Extract the **automation.tar.gz** file using the following command:

```
tar -xvf automation.tar.gz
```

a. Run the following commands to create the `plugin-registry` binary build, build the image using the dynamic plugin directory, and deploy the resulting image:

Where `$DYNAMIC_PLUGIN_ROOT_DIR` is the local directory containing the dynamic plugin files.

```
export DYNAMIC_PLUGIN_ROOT_DIR=/home/lab-user
```

```
oc new-build httpd --name=plugin-registry --binary
```

```
oc start-build plugin-registry --from-dir=$DYNAMIC_PLUGIN_ROOT_DIR --wait
```

```
oc new-app --image-stream=plugin-registry
```

b. Go to OpenShift Web Console, navigate to the `portal` project, and Then Workloads → Topology verify that the `plugin-registry` pod is running successfully.

Create OpenShift Secrets for AAP Authentication

OpenShift Secrets are secure, encrypted storage for sensitive data. We'll create a secret that holds the OAuth credentials and API token you generated in Tasks 1 and 3. The Automation Portal Helm chart will reference this secret during deployment.

1. Go to Workloads > Secrets in the OpenShift console and make sure the project `portal` is selected. You should create the secret `secrets-rhaap-portal` with the below information. You can also

create the secret using the `oc` command line tool.

Before running the secret creation command, gather the values from Tasks 1 and 3:

- `<aap-instance-url>` — The URL of your AAP instance (e.g., `https://aap.example.com`)
- `<oauth-client-id>` — The Client ID from the previous task.
- `<oauth-client-secret>` — The Client Secret from the previous task.
- `<aap-token>` — The API Token from the previous task.

1. Run the following command, replacing the placeholders with your actual values:

```
oc create secret generic secrets-rhaap-portal \
  --from-literal=aap-host-url="<your-aap-url>" \
  --from-literal=oauth-client-id="your-client-id-here" \
  --from-literal=oauth-client-secret="your-client-secret-here" \
  --from-literal=aap-token="your-api-token-here" \
  -n portal
```

The secret name (`secrets-rhaap-portal`) is important—the Helm chart expects this exact name. If you change it, you'll need to update your Helm values during deployment.

Verify the Secret Was Created

Confirm the secret exists:

```
oc get secret secrets-rhaap-portal -n portal
```

You should see output like:

```
NAME                    TYPE     DATA   AGE
secrets-rhaap-portal    Opaque   4      10s
```

Once the above is done. You're now ready for the next phase: deploying the Automation Portal via Helm chart!

Hands-on Lab: Installing and Verifying the Automation Portal on OCP

In this lab, you will deploy the Automation Portal on your OpenShift Container Platform (OCP) environment.

Installing the Automation Portal

1. Go to Ecosystem > Software Catalog in the OpenShift web console.
2. Search for "Automation Portal" in the catalog and select it.
   a. Click on **Create** to start the installation process.
   b. Change the **clusterRouterBase** to the correct value for your cluster. For example, if the OCP cluster url is `https://console-openshift-console.apps.cluster-kstzt.dyn.redhatworkshops.io` then the clusterRouterBase should be `apps.cluster-kstzt.dyn.redhatworkshops.io`.
   c. Change the Release Name to `automation-portal` and you can change it if you want.
   d. By default, it will be setting up at the Organization **Default** for the automaton portal the parameter is orgs if you wanted to change it.

Once the above settings are done, click on **Create** to start the installation process.

The installation process will take a few minutes. You can monitor the progress Workloads > Topology in the portal project. Once the installation is complete, you will see the Automation Portal components running.

1. Click on **automation-portal-rhaap-portal** resource to view the details. You will find the route URL in the **Routes** section. This URL is used to access the Automation Portal web interface.
2. Copy the route URL and go to Ansible Automation Platform > Access Management > OAuth Applications.
3. Edit the automation-portal OAuth application to include the route URL as a valid redirect URI and save the changes. This step is crucial for the OAuth authentication to work correctly.

```
https://<aap-portal-hostname>/api/auth/rhaap/handler/frame
```

1. Access the Web UI of the Automation portal

== Task 3: Configuring Initial RBAC Navigation

To reduce UI complexity for end-users, we will implement role-based menu filtering.

1. **Log in as an Administrator.**
2. Navigate to **Administration** → **Access** → **User Interface**.
3. Create a "Standard User" role policy:
   - **Target:** User Group: `Developers`
   - **Restrict Navigation:** Select only `Templates` and `Jobs`.
4. **Validate:**
   - Open an incognito browser window.

- Log in as a user belonging to the `Developers` group.
- Confirm the sidebar reflects only the permitted items, successfully reducing the cognitive load for the end-user.

== Troubleshooting Tips * **Image Pull Errors:** Ensure your OpenShift cluster has the necessary pull secrets configured for the Red Hat Registry if you are using disconnected installation methods. * **Service Connectivity:** If the portal cannot sync with the Hub or Controller, check the network policies in the `ansible-automation-platform` namespace to ensure egress traffic is permitted between these components. * **Logs:** If the UI fails to load, inspect the controller logs: `oc logs deployment/aap-portal-controller -n ansible-automation-platform`

== Visual Execution Environment Builder

Visual Execution Environment Builder

The Visual Execution Environment (EE) Builder is a GUI-based tool that simplifies the creation of portable, reproducible automation runtime images. It replaces complex CLI-based workflows by providing a guided interface to manage the assembly of `ansible-core`, collection dependencies, Python packages, and system libraries.

Key features include:

- **Guided Workflow**: Utilizes a step-by-step wizard to define custom EEs, removing the need to manually author `execution-environment.yml` files.
- **Integrated Discovery**: Enables users to select Ansible Content Collections directly from the centralized discovery catalog.
- **Definition Management**: Automatically generates complete EE definition files with options to download or publish directly to repositories.
- **CI/CD Integration**: Supports automated container image builds through scaffolded CI pipelines and integration with Git-based workflows.
- **Customization**: Allows for the definition of pre-build and post-build steps, alongside the integration of Red Hat domain-specific presets.

=== Architecture and transition from manual CLI workflows to GUI-based building

Architecture and Transition from Manual CLI Workflows to GUI-Based Building

In Ansible Automation Platform (AAP) 2.7, the shift from manual command-line interface (CLI) workflows to the Visual Execution Environment (EE) Builder represents a fundamental change in how platform engineers manage automation dependencies. This transition prioritizes repeatability, governance, and user accessibility by abstracting the complexities of container image construction.

The Architectural Shift

Traditionally, building Execution Environments required a manual, multi-step CLI workflow

using `ansible-builder`. This approach often involved:

1. **Manual YAML Authoring:** Creating `execution-environment.yml` definition files by hand.
2. **CLI Orchestration:** Executing `ansible-builder build` commands within a terminal.
3. **Disconnected Tooling:** Managing container registries, Git repositories, and dependency files as separate, siloed processes.
4. **Error-Prone Environments:** Inconsistencies in local environment configurations leading to "it works on my machine" syndromes.

The new **Visual EE Builder** architecture moves these processes into the Automation Portal. It functions as an orchestration layer that interfaces directly with your container registry and version control systems. By moving the logic into the portal, AAP enforces consistent build parameters, integrates with the platform's RBAC policies, and provides a centralized audit trail for every image built.

Benefits of the GUI-Based Approach

Transitioning to a GUI-based builder offers several architectural advantages:

- **Standardization:** Instead of disparate scripts, the portal uses a guided wizard that ensures all necessary metadata (Python packages, system libraries, and collection dependencies) is captured in a standardized format.
- **Integrated Dependency Management:** The builder leverages the **Centralized Content Discovery** catalog. Rather than manually typing collection requirements, you browse the synced catalog, ensuring you are using verified, organizationally approved content.
- **Abstraction of Complexity:** Platform engineers no longer need to maintain complex `ansible-builder` logic. The portal handles the underlying container build instructions while presenting a clean, intent-based interface.
- **Automated CI/CD Integration:** The Visual Builder does not just build an image—it scaffolds the entire lifecycle. By publishing definitions to Git and automating pipelines via GitHub Actions, the transition moves from "manual execution" to "automated lifecycle management."

Transition Strategy: From CLI to GUI

When migrating your workflows from manual CLI processes to the Visual EE Builder, follow this structured approach to ensure organizational continuity:

Phase	Action
Discovery	Sync existing collections and base images into the portal using the Content Discovery feature.
Mapping	Identify existing `execution-environment.yml` files and map their contents (bindep, requirements.txt, collections) to the GUI fields in the EE Builder wizard.

Phase	Action
Scaffolding	Use the Visual Builder to generate the initial definition, then utilize the "Export to Git" feature to establish a version-controlled source of truth.
Automation	Replace local CLI build triggers with the scaffolded CI pipeline (GitHub Actions/GitLab CI) to ensure consistent image promotion.

Hands-On: Evaluating your current EE Build process

To prepare for the transition, perform the following audit of your current environment:

1. **Audit Current Definitions:** Gather all existing `execution-environment.yml` files currently used by your teams.
2. **Verify Content Sources:** Ensure all dependent Ansible Content Collections are available in your Private Automation Hub or connected external registry.
3. **Pilot Migration:** Select one non-critical Execution Environment. Open the **Visual EE Builder** in your AAP 2.7 portal and attempt to replicate the `execution-environment.yml` using the GUI wizard.
4. **Validate Output:** Compare the resulting container image metadata with your original CLI-built image to confirm parity in system libraries and Python dependencies.

By centralizing EE builds in the portal, you regain control over the "What" and "How" of your automation runtime. Every GUI-based build is tracked and gated by admin-defined access policies, effectively turning your EE creation process into a governed, transparent activity.

=== Utilizing the step-by-step wizard for custom EE creation

Utilizing the Step-by-Step Wizard for Custom EE Creation

Creating Execution Environments (EEs) traditionally required manual management of `execution-environment.yml` files, complex dependency resolution, and deep knowledge of container build contexts. The Visual Execution Environment Builder within the Automation Portal abstracts this complexity, providing a guided, wizard-driven interface to ensure standardized, error-free images.

Overview of the Wizard Workflow

The wizard workflow is designed to move from high-level intent to a concrete container definition. By leveraging Red Hat-supported presets, the wizard minimizes the "guesswork" involved in selecting base images and dependency formats.

The process follows a logical progression: 1. **Definition Naming and Scoping:** Defining the EE purpose and metadata. 2. **Base Image Selection:** Choosing from Red Hat-optimized, pre-validated base images (e.g., `ee-minimal-rhel9`, `ee-supported-rhel9`). 3. **Dependency Injection:** Declarative selection of Ansible Content Collections, Python packages (`requirements.txt`), and

system-level libraries (`bindep.txt`). 4. **Build Customization:** Defining pre-build and post-build shell commands to handle non-standard configuration tasks. 5. **Review and Validation:** Real-time generation of the underlying YAML definition.

Integrating Presets and Dependencies

The wizard leverages Red Hat domain-specific presets to ensure security and compliance. When you select a base image, the wizard automatically pulls the relevant metadata, preventing common version mismatches between the OS layer and the Ansible Automation Platform engine.

- **Ansible Content Collections:** Users can search and select collections directly from the catalog. The wizard manages the `requirements.yml` structure, ensuring all dependencies are pinned correctly.
- **System Libraries:** Rather than manually crafting complex `Dockerfile` layers, users add system packages via a simple list interface, which the wizard translates into an optimized `bindep.txt` configuration.

The EE definition workflow generates a reusable Backstage software template (e.g., `<ee-name>-template.yaml`). This allows organizations to move from one-off EE creation to a "Golden Image" catalog, where team members can provision standardized environments based on pre-approved templates.

Hands-on Lab: Reviewing the Generated Definition

In this exercise, we will examine the output of the EE Builder to understand how the GUI translates user input into machine-readable automation code.

Prerequisites

- Access to the Automation Portal on OpenShift.
- An active session within the Visual EE Builder wizard.

Steps

1. **Navigate to the Review Screen** Complete the input fields for your desired EE (Base image, collections, and dependencies). Proceed to the final step of the wizard labeled "Review."
2. **Examine the Generated YAML** Observe the auto-generated YAML manifest. Verify the following:
   - **Base image:** Confirm it matches the selected RHEL-based image.
   - **Dependency declarations:** Check that your selected collections and Python packages are accurately represented in the `dependencies` block.
   - **Syntax integrity:** Note the automatic handling of indentation and list structure, which prevents common "whitespace" errors associated with manual YAML authoring.
3. **Comparative Analysis** Compare this generated YAML with a manual `execution-environment.yml` file.

- **Self-Reflect:** How did the wizard handle the collection source references?
- **Observation:** The wizard effectively handles the mapping of external Galaxy/Automation Hub URLs, abstracting the need for manual URL management.

4. **Prepare for Publication** [IMPORTANT] Ensure your configuration is correct, but **do not click "Publish" yet**. In the next lab section, we will integrate this definition with Git and trigger the automated CI pipeline.

Best Practices for Wizard Usage

- **Use Version Pinning:** Always specify versions for collections and Python packages within the wizard to ensure reproducible builds.
- **Leverage Post-Build Steps:** Use the post-build section for cleanup tasks or custom environment variable injection rather than manual Dockerfile modifications.
- **Catalog Early:** Once your EE definition is stable, export the generated Backstage template to your private catalog to enable self-service consumption by other developers.

=== Integrating Red Hat domain-specific presets and base container images

Integrating Red Hat Domain-Specific Presets and Base Container Images

In the Ansible Automation Platform 2.7 Visual Execution Environment (EE) Builder, selecting the correct foundation is the most critical step in ensuring security, compliance, and performance. By leveraging Red Hat's domain-specific presets, architects can bypass the "blank slate" problem and inherit validated configurations tailored to specific operational domains.

Understanding Domain-Specific Presets

Domain-specific presets are curated configuration templates provided by Red Hat. They include pre-configured collections, Python dependencies, and system-level libraries optimized for specific use cases.

- **Networking:** Includes `ansible.netcommon`, `cisco.ios`, and `arista.eos` collections, alongside system-level utilities for network device communication.
- **Security:** Focused on compliance auditing and vulnerability management, featuring `ansible.posix` and various security-hardened utility packages.
- **Cloud:** Pre-loaded with cloud-provider-specific SDKs (AWS, Azure, GCP) and the corresponding `community.cloud` or `amazon.aws` collections.

Using these presets ensures that your EEs adhere to organizational standards for library versions and security patching, reducing the maintenance overhead of managing disparate `execution-environment.yml` files.

Base Container Images and Optimization

The EE Builder utilizes Red Hat Universal Base Image (UBI) 9 as the foundation. When you integrate a preset, the builder automatically references the associated UBI-based image from

the Red Hat Ecosystem Catalog.

When operating in disconnected environments, ensure your local registry is mirrored correctly using `skopeo copy`. As noted in the platform documentation, the use of SHA256 digests is recommended to ensure that your custom EE builds remain immutable and consistent across development and production OCP clusters.

Hands-on Activity: Creating an EE from a Domain Preset

This activity walks you through creating an EE using the Visual Builder's preset engine.

Prerequisites

- Access to the Automation Portal on OpenShift.
- `skopeo` configured to mirror images from `registry.redhat.io` if operating in a disconnected cluster.

Step 1: Initialize the Visual Builder Wizard

1. Navigate to the **Execution Environments** tab in your Automation Portal.
2. Click **Create New Environment**.
3. Select **"Use a Domain-Specific Preset"** from the wizard options.

Step 2: Selecting and Configuring the Preset

1. Choose the **Networking** preset from the available catalog.
2. The UI will auto-populate the `execution-environment.yml` definition with:
   - The standard Red Hat UBI 9 base image.
   - Recommended networking-related Ansible Collections.
   - Required system libraries (e.g., `libssh`).
3. Click **Next** to proceed to the customization stage.

Step 3: Managing Dependencies

In the "Dependencies" pane, you can augment the preset: 1. Under **Ansible Collections**, add any specific vendor-proprietary collections your environment requires. 2. Under **Python Packages**, add specific `pip` requirements. * **Example:** Add `netaddr` or `paramiko` if not already included in the preset base. 3. Define **Pre-build steps** (e.g., `RUN dnf install -y my-custom-tool`) if you need to install binary dependencies not covered by the preset.

Step 4: Verification

1. Review the generated definition file.
2. Click **Build** to trigger the CI pipeline.
3. Monitor the build logs via the link provided in the UI; verify that the `base_image` reference resolves to your internal registry (e.g., `mirror.example.com/ansible-automation-platform-27/ee-supported-rhel9`).

Troubleshooting

- **Image Pull Backoff:** If the build fails, check that your `imageRegistry` configuration in the Helm `values.yaml` is set to the registry host only, without repository paths. The builder will append the default path automatically.
- **Dependency Conflicts:** If custom packages conflict with the preset, prioritize using `ansible.cfg` to set `collections_path` or use the "Overrides" feature in the EE Builder to pin specific versions, overriding the preset defaults.

=== Managing dependencies: Ansible Content Collections, Python packages, and system libraries

Managing Dependencies in Execution Environments

In the context of Ansible Automation Platform 2.7, managing dependencies is critical for ensuring that Execution Environments (EEs) are portable, predictable, and secure. The Visual Execution Environment Builder simplifies this process by replacing error-prone manual CLI workflows with a declarative, GUI-based dependency management interface.

Understanding EE Dependency Architecture

An Execution Environment is a container image that packages the Ansible Core engine, Python interpreter, Ansible Content Collections, and necessary system-level libraries. To maintain a "single source of truth," the Visual Builder categorizes dependencies into three distinct layers:

- **Ansible Content Collections:** The functional modules and plugins required for your automation tasks.
- **Python Packages:** The language-specific libraries required by your collections or custom scripts (e.g., `netaddr`, `jmespath`).
- **System Libraries (RPMs):** OS-level binaries required by the underlying container environment to support execution (e.g., `jq` for JSON processing, `openssh-clients` for connectivity).

The "Browse-Before-Build" Workflow

To prevent dependency drift and redundant development, AAP 2.7 introduces a **Browse-Before-Build** pattern. Before defining an EE, developers should consult the Centralized Content Discovery catalog. This ensures that you are utilizing vetted, existing collections rather than importing duplicate or unapproved versions from external repositories.

Hands-on Lab: Configuring EE Dependencies

In this activity, you will configure the dependency manifest for a custom Execution Environment using the Visual Builder.

Prerequisites

- Access to the Ansible Automation Platform 2.7 Web UI.
- Permissions to create and edit Execution Environment definitions.

Step 1: Adding Ansible Content Collections

Collections must be sourced from your defined Content Discovery catalogs to ensure integrity.

1. Navigate to the **Visual EE Builder** and create a new definition.
2. In the **Collections** section, select **Add from Catalog**.
3. Search for the required collections (e.g., `ansible.netcommon` or `cisco.ios`).
4. Specify the version constraints to ensure consistency across your team's environments.

Step 2: Defining Python and System Packages

Once the primary collections are defined, you must satisfy their underlying dependencies.

1. Proceed to the **Dependencies** screen within the builder.
2. **Add Python Packages:** Add the requirements needed by your automation scripts.
   - Input: `netaddr`, `paramiko`, `jmespath`
3. **Add System Packages (RPMs):** Add the necessary Linux binaries.
   - Input: `jq`, `openssh-clients`
4. Click **Next** to validate the definition.

Step 3: Verification

The Visual Builder will resolve the dependency tree.

- **Expected Result:** The system validates that all requested Python and system packages are compatible with your base image. You should see a consolidated summary listing the collections, Python dependencies, and RPMs ready for the build process.

Best Practices for Dependency Management

- **Version Pinning:** Always specify exact versions for Python packages and Collections. Relying on "latest" can lead to unpredictable CI/CD pipeline failures when upstream packages release breaking changes.
- **Minimalism:** Only include the libraries required for the specific task. Over-bloating an EE with unnecessary system packages increases the attack surface and slows down deployment times.
- **CI Integration:** Once your dependencies are configured in the GUI, export the definition to

Git. This allows your CI pipeline to trigger automated builds whenever the `execution-environment.yml` file is updated, ensuring the infrastructure-as-code (IaC) lifecycle is maintained.

=== Defining custom pre-build and post-build steps

Defining Custom Pre-Build and Post-Build Steps

In the context of the Visual Execution Environment (EE) Builder, automation engineers often require specific system-level configurations that go beyond standard Ansible Content Collection or Python package dependencies. Custom build steps allow you to inject arbitrary commands into the container image build process, ensuring your Execution Environment meets specific organizational security, compliance, or tooling requirements.

Architectural Overview

Custom build steps are executed during the container build lifecycle, specifically interacting with the `ansible-builder` process under the hood. By utilizing the GUI-based Visual Builder, you can define these operations without manually editing `execution-environment.yml` files.

- **Pre-build steps:** Executed before the installation of Python packages and Ansible Collections. These are ideal for configuring system repositories, installing OS-level development headers, or preparing the file system.
- **Post-build steps:** Executed after the installation of dependencies. These are typically used for cleanup operations, generating metadata files (such as build timestamps or versioning logs), or running security scans/validations on the final image state.

Configuring Build Steps in the Visual Builder

The Visual Builder provides an intuitive interface to insert these commands directly into the container definition.

Best Practices for Custom Steps

- **Idempotency:** Ensure that your commands are idempotent and exit with a non-zero status code if a failure occurs to prevent the creation of broken container images.
- **Minimize Layers:** Where possible, chain commands using `&&` to reduce the number of layers in the resulting container image, which optimizes image size and build speed.
- **Security:** Avoid echoing sensitive credentials or environment variables in build steps, as these may be persisted in the container's history.

Hands-on Lab: Defining Custom Build Steps

In this exercise, you will inject custom metadata into your Execution Environment to track build provenance.

Exercise 2.5: Define Custom Build Steps (Optional)

Ensure you have completed the prerequisite setup of your project workspace within the Visual EE Builder before beginning this task.

1. **Navigate to the Build Steps screen:** In the Visual Builder wizard, progress to the "Custom Build Steps" configuration screen.
2. **Define the Post-Build command:** Locate the input field for "Post-build commands." Enter the following command to create a build information file:

```
[source,bash]
----
RUN echo "Network Automation EE - Built $(date)" > /etc/ee-build-info
----
```

3. **Validate the definition:** Observe the definition preview window. The builder will automatically format the command for inclusion in the underlying build definition files.
4. **Verification:** Click **Next** to save your configuration. Once the image build completes, you can verify the existence of the file by running:

```
[source,bash]
----
podman run --rm <your-ee-image-name> cat /etc/ee-build-info
----
```

If your build fails at the custom step, review the build logs in the Automation Portal interface. The logs capture `stdout` and `stderr` from your custom commands, providing immediate troubleshooting feedback.

=== Publishing definitions to Git and automating CI pipelines with GitHub Actions

Publishing and Automating Execution Environment (EE) Pipelines

In modern automation workflows, the Execution Environment (EE) definition should not exist solely within the portal. To achieve "Automation as Code" best practices, definitions must be externalized to version control systems. This ensures auditability, allows for pull-request-based reviews, and enables automated CI/CD workflows.

Architectural Benefits of Git Integration

By publishing EE definitions to Git, you transition from a "state-based" model within the portal to a "source-of-truth" model. This provides several key advantages:

- **Version History:** Every change to your dependencies (collections, python packages, or system libraries) is tracked via Git commits.
- **Pipeline Automation:** GitHub Actions can be triggered automatically upon a push,

building the image and pushing it to a container registry like Quay.io.

- **Reproducibility:** If an EE is deleted from the portal, it can be fully reconstructed by importing the definition back from the repository.

Defining the CI/CD Pipeline

When publishing via the Automation Portal's Visual Builder, the system doesn't just push configuration files; it scaffolds a production-ready CI pipeline. The generated pipeline typically performs the following steps:

1. **Checkout:** Pulls the EE definition files from the repository.
2. **Authentication:** Uses stored secrets to log into your container registry (e.g., Quay.io).
3. **Build:** Uses the `ansible-builder` logic to resolve dependencies and build the container image.
4. **Push:** Uploads the newly tagged container image to the target registry for use in your Ansible Automation Platform clusters.

Hands-on Lab: Publishing and CI Scaffolding

This exercise guides you through moving your EE definition to GitHub and enabling automated builds.

Exercise 3.1: Publish the EE Definition to GitHub

1. Navigate to the **EE Builder** in the Automation Portal and select your saved EE definition.
2. On the review/output screen, select **Publish to Git**.
3. Configure the following Git publishing options:
   - **Repository:** Select `Create new repository`.
   - **Branch:** `main`.
   - **Org/User:** Enter your `<your-github-username-or-org>`.
   - **Repo name:** `network-automation-ee`.
   - **Authentication:** Enter your GitHub Personal Access Token (PAT) with `repo` scope.
4. Click **Publish**. Wait for the confirmation message verifying the repository creation and file push.

Exercise 3.2: Scaffold a GitHub Actions CI Pipeline

1. Within the publication workflow, select the option to **Include GitHub Actions workflow**.
2. Configure the build pipeline parameters:
   - **Target registry:** `quay.io/<your-namespace>`
   - **Image name:** `network-automation-ee`
   - **Registry credentials:** Provide your container registry credentials (these are stored as repository secrets in GitHub).

3. Click **Publish** or **Create**.
4. Once completed, navigate to your new GitHub repository. Under the **Actions** tab, you will see the scaffolded workflow ready for its first execution.

To maintain security, ensure your `GITHUB_PAT` and `REGISTRY_CREDENTIALS` are managed as **Repository Secrets** in GitHub. Never commit plain-text credentials to your EE definition repository.

Integration with Backstage Software Templates

Once the pipeline is established, you can import the generated repository as a **Backstage Software Template**. This allows other teams in your organization to discover your validated EE definition, "fork" or "clone" it via the portal, and instantly gain a pre-configured CI pipeline. This promotes organizational standardization by ensuring every team uses the same build patterns for their network or application automation environments.

=== Importing and managing reusable Backstage software templates

Importing and Managing Reusable Backstage Software Templates

In the Ansible Automation Platform 2.7 ecosystem, the Visual Execution Environment (EE) Builder does more than just generate container images; it facilitates organizational standardization through Backstage software templates. When an EE definition is finalized, the system generates a `<ee-name>-template.yaml` file. This file acts as a scaffold, capturing the base image, collections, and dependency requirements.

By importing these templates into the Automation Portal, administrators can transform a one-time build into a reusable baseline, allowing different teams to adopt "golden" configurations for their automation infrastructure.

Architectural Benefits of Reusable Templates

- **Standardization:** Ensures all teams build EEs using approved base images and vetted Ansible Collections.
- **Reduced Complexity:** New users can start with a pre-configured template rather than defining complex dependencies from scratch.
- **Version Control Integration:** Templates stored in Git track changes to your automation requirements over time.
- **Self-Service Enablement:** Enables a "browse-before-build" workflow where developers can discover and deploy existing, compliant EE configurations.

Managing Templates in the Portal

Managing these templates involves a lifecycle of analysis, validation, and publication. When you import a template, the Automation Portal parses the YAML metadata to ensure it complies with your organization’s security and architecture policies.

Hands-on Lab: Importing a Reusable Template

This lab assumes you have already generated an EE definition and pushed the resulting `network-automation-ee-template.yaml` to your GitHub repository.

Prerequisites

- Access to the Automation Portal with `Administrator` or `Template Manager` permissions.
- A valid raw URL to your `template.yaml` file hosted in a reachable GitHub repository.

Steps

1. **Locate the Template Source:** Navigate to your GitHub repository and locate the generated file (e.g., `network-automation-ee-template.yaml`).
2. **Retrieve the Raw URL:** Open the file in the GitHub web interface and click the **Raw** button to get the direct file URL (e.g., `https://raw.githubusercontent.com/<your-username>/network-automation-ee/main/network-automation-ee-template.yaml`).
3. **Initiate Import:** Log in to the Automation Portal. In the top-right navigation, click **Add Template**.
4. **Define Source:** Select the **Select URL** option from the provided methods.
5. **Validation:** Paste the raw URL copied in step 2 and click **Analyze**.

The "Analyze" phase is critical. The portal validates the YAML syntax and ensures the referenced base image and collections are compatible with your current Automation Platform version.

+ 6. **Review:** Inspect the parsed template data in the UI. Ensure the base image, specific Python packages, and Ansible Collections align with your team's requirements. 7. **Finalize:** Click **Import** to register the template in the portal catalog.

Troubleshooting Template Imports

If the import process fails during the "Analyze" phase, check the following:

- **Connectivity:** Ensure the GitHub repository is public or that the Automation Portal has the necessary credentials (SSH/PAT) to access private repositories.
- **YAML Schema:** Verify that the `template.yaml` file has not been manually edited in a way that violates the Backstage template schema required by the Automation Portal.
- **Dependency Resolution:** If the analysis indicates "Unresolved Dependencies," ensure that any custom collections listed in the template are available in your private Automation Hub or via the configured global Galaxy source.

=== Hands-on Lab on Syncing content sources and cataloging collections and Building a custom Execution Environment using the Visual Builder

Hands-on Lab: Content Discovery and Visual EE Building

In this lab, you will leverage the Automation Portal to unify disparate automation sources and

transition from manual image creation to a streamlined, GUI-based Execution Environment (EE) workflow.

Prerequisites

- Access to an OpenShift Container Platform (OCP) cluster with AAP 2.7 installed.
- An authenticated session in the Automation Portal.
- Read/Write access to a connected Git repository.

Task 1: Syncing Content Sources and Cataloging

Before building an EE, we must aggregate your automation assets into a unified catalog.

1. **Navigate to Discovery:** In the Portal sidebar, select **Content Discovery** > **Sources**.
2. **Add Sources:** Click **Add Source**.
   - Select `GitHub` and provide your organization URL and token.
   - Select `Private Automation Hub` and provide the URL and API token.
3. **Initiate Synchronization:** Click **Sync Now** on both entries.
4. **Verify Catalog:** Navigate to the **Catalog** tab. You should see Ansible Content Collections aggregated from both GitHub and your Private Automation Hub.
5. **Apply Governance:** Use the **Filter** menu to apply an "Approved" tag, ensuring that only verified collections appear in your EE builder selection list.

Task 2: Building a Custom Execution Environment

We will now use the Visual EE Builder to generate a container definition without writing a single line of YAML or `ansible-builder` configuration files.

1. **Access the Builder:** Navigate to **Execution Environments** > **Visual Builder**.
2. **Initialize Definition:** Click **Create New Definition**. Provide a name (e.g., `network-ops-ee`) and a base container image (e.g., `registry.redhat.io/ansible-automation-platform-27/ee-minimal-rhel9`).
3. **Add Collections:**
   - Click **Add Collection**.
   - The portal will display your unified catalog from Task 1.
   - Search for `cisco.ios` and `arista.eos`. Select them and click **Add to Definition**.
4. **Inject Dependencies:**
   - Click the **Dependencies** tab.
   - In the **Python Requirements** section, add `netmiko` and `ncclient`.
   - In the **System Packages** section, add `libssh-devel`.
5. **Define Build Steps:**
   - Navigate to **Custom Steps**.

- Add a **Pre-build** step: Enter a shell command to verify the file system structure.
- Add a **Post-build** step: Include a command to run a security scan script (e.g., `ansible-lint -v`).

6. **Finalize and Publish:**
   - Review the generated preview.
   - Click **Save & Publish**. Select your GitHub repository; the builder will automatically push the `execution-environment.yml` and associated metadata to your repository.

Troubleshooting Tips

- **Sync Failures:** If sources fail to sync, check the **Activity Log** in the Portal to verify API token permissions and network connectivity to the remote source.
- **Dependency Conflicts:** If a collection requires a specific Python version not supported by your base image, the builder will provide a validation error. Switch to a more compatible base image (e.g., `ee-supported-rhel9`).
- **RBAC Restrictions:** If you cannot see the Visual Builder, ensure your user account is assigned the `Automation Administrator` or `EE Builder` role.

Verification

Verify that the build process has been triggered by navigating to your linked GitHub repository. You should see a `build/` directory populated with your definition, confirming that the Visual Builder has successfully handed off the configuration to your CI/CD pipeline.

=== Hands-on Lab Deploying a scaffolded CI pipeline for automated container image builds

Hands-on Lab: Deploying a Scaffolded CI Pipeline for Automated EE Builds

In this lab, you will transition from manual Execution Environment (EE) construction to an automated DevOps workflow. By leveraging the Visual Execution Environment Builder, you will scaffold a GitHub repository pre-configured with a CI pipeline, enabling automated container builds and registry pushes without manual CLI intervention.

Prerequisites

- Access to an instance of the Ansible Automation Platform 2.7 Automation Portal.
- An active GitHub account.
- A target container registry (e.g., Quay.io) with valid credentials.
- A pre-defined EE configuration within the Visual Builder.

Exercise: Scaffolding a CI Pipeline

This exercise demonstrates how to move from a defined EE specification to an automated delivery pipeline.

Step 1: Select the CI Pipeline Scaffolding Option

After you have finalized your EE configuration (dependencies, collections, and base images) within the Visual Execution Environment Builder:

1. Navigate to the **Output Options** screen of the wizard.
2. Select the option labeled **"Scaffold a GitHub repository with CI pipeline"**.

This option triggers the builder to generate not only the `execution-environment.yml` but also the necessary CI workflow files (e.g., `.github/workflows/build-ee.yml`) required for automated builds.

Step 2: Configure Pipeline Parameters

The builder requires connection details to automate the push of your container image once the build succeeds.

1. **Target Registry:** Enter your container registry URL (e.g., `quay.io/<your-namespace>`).
2. **Image Name:** Specify the tag name for your environment (e.g., `network-automation-ee`).
3. **Registry Credentials:** When prompted, provide the necessary secrets to allow GitHub Actions to authenticate with your container registry.
4. Click **Publish/Create**.

Step 3: Verify Repository Creation

Once the process completes:

1. Navigate to the GitHub repository URL provided by the builder.
2. Observe the generated file structure:
    - `execution-environment.yml`: The declarative definition of your EE.
    - `.github/workflows/`: Contains the pre-configured GitHub Actions workflow.
3. Open the workflow file to review the build steps:
    - **Build Stage:** Utilizes the EE definition to build the container image.
    - **Push Stage:** Authenticates with the registry defined in Step 2 and pushes the image.

Step 4: Trigger and Monitor the Build

To ensure the pipeline is functional:

1. Navigate to the **Actions** tab in your newly created GitHub repository.
2. If the build did not trigger automatically upon creation, manually trigger the workflow via the **"Run workflow"** button.
3. Monitor the build logs to ensure the environment dependencies (Ansible Collections, Python packages) are installed correctly and the resulting image is pushed to your registry.

Troubleshooting Tips

- **Authentication Failures:** Ensure that the secrets stored in the GitHub repository (Settings > Secrets and variables > Actions) correspond to the credentials used for your container registry.
- **Build Timeouts:** If the build fails due to large dependency lists, check the `execution-environment.yml` for any extraneous libraries that can be optimized.
- **Registry Visibility:** Confirm that your target registry namespace is set to public, or that the registry credentials have sufficient write permissions for the specified repository.

Summary

By completing this lab, you have successfully moved from a manual, CLI-heavy process to a **GitOps-driven lifecycle**. Your execution environment is now version-controlled, and any future changes to your `execution-environment.yml` will automatically trigger a rebuild and push to your container registry, ensuring your automation infrastructure remains consistent and up-to-date.

== Centralized Content Discovery

Centralized Content Discovery

Centralized Content Discovery provides a unified catalog architecture designed to aggregate automation assets from distributed sources into a single, searchable interface. It eliminates manual discovery efforts by streamlining how platform engineers manage and surface Ansible Content Collections.

Key features include:

- **Multi-Source Aggregation:** Integrates diverse repositories, including GitHub, GitLab, and Private Automation Hub.
- **Continuous Synchronization:** Supports scheduled updates to ensure the catalog remains current with the latest automation assets.
- **Controlled Visibility:** Implements predefined scoping and access policies to manage how users browse and discover available collections.
- **Simplified Traceability:** Enables a "browse-before-build" workflow, allowing developers to identify existing assets and avoid redundant development.

=== Unified catalog architecture for multi-source aggregation

Unified Catalog Architecture for Multi-Source Aggregation

In modern enterprise automation, content is rarely stored in a single location. Ansible Collections are often dispersed across distributed version control systems (like GitHub or GitLab) and local distribution points (like Private Automation Hub). The **Unified Catalog Architecture** in Ansible Automation Platform 2.7 solves the "siloed content" problem by abstracting the storage layer and providing a single pane of glass for discovery.

Architectural Overview

The Unified Catalog operates as a middleware layer between the automation consumer and the various content repositories. Instead of forcing developers to hunt through multiple URLs, the portal acts as a metadata aggregator.

```
@startuml
package "Content Sources" {
  [GitHub Organization] as GH
  [Private Automation Hub] as PAH
  [GitLab Instance] as GL
}

package "Automation Portal" {
  [Sync Engine]
  [Metadata Database]
  [Unified API/UI]
}

GH --> [Sync Engine] : Poll/Webhook
PAH --> [Sync Engine] : REST API
GL --> [Sync Engine] : Poll
[Sync Engine] --> [Metadata Database]
[Metadata Database] --> [Unified API/UI]
@enduml
```

Key Components

1. **The Sync Engine:** A background service that periodically connects to configured remote sources. It performs a lightweight metadata crawl to identify available Collections, their versions, and documentation.
2. **Metadata Database:** A persistent storage layer within the portal that caches the state of discovered collections. This allows for near-instant search results without needing to reach out to external APIs during every UI query.
3. **Abstraction Layer:** Normalizes the data structure from diverse sources (e.g., standardizing how a GitHub-hosted collection version is represented versus an Automation Hub-hosted collection).

Benefits of Aggregation

- **Reduced Discovery Friction:** Developers can search for a module or collection without needing to know the hosting infrastructure beforehand.
- **Version Visibility:** The catalog tracks metadata across sources, allowing users to compare if an internal, customized collection in the Private Automation Hub is newer or older than an upstream version in GitHub.
- **Policy-Driven Access:** By aggregating content, administrators can apply RBAC to the catalog, ensuring that users only see collections they are authorized to consume, regardless

of the source.

Hands-on Activity: Configuring Multi-Source Aggregation

In this activity, you will configure the Automation Portal to aggregate content from a GitHub organization and a Private Automation Hub.

Prerequisites

- Access to an OCP cluster with Automation Portal installed.
- A GitHub Personal Access Token (PAT) with `repo` scope.
- The API endpoint and token for your Private Automation Hub.

Step 1: Define Content Sources

1. Navigate to **Content Management** > **Sources** in the Automation Portal.
2. Click **Add Source**.
3. For the **GitHub Source**:
   - Select `GitHub` as the source type.
   - Input your organization URL: <code><a href="https://github.com/<your-org>" class="bare">https://github.com/<your-org>;</code>.
   - Provide the PAT in the credential field.
4. For the **Private Automation Hub Source**:
   - Select `Private Automation Hub`.
   - Provide the `https://automation-hub.example.com` URL.
   - Provide the API token for authentication.

Step 2: Triggering Sync

1. Once configured, navigate to the **Sync Settings**.
2. Set the **Sync Schedule** to every 6 hours (recommended for medium-sized environments).
3. Click **Sync Now** to pull the initial metadata from both locations.

Step 3: Verifying the Unified Catalog

1. Navigate to the **Catalog** tab in the main navigation menu.
2. Observe that collections from both GitHub and the Private Automation Hub are present.
3. **Verification:** Select a specific collection. In the details pane, locate the **Origin** field. It should clearly display the source (e.g., `Source: Private Automation Hub` or `Source: GitHub`).

If a collection exists in both sources, the portal will surface both versions. Use the "Default Version" settings in the portal to pin the preferred version for your automation teams.

=== Configuring supported sources including GitHub, GitLab, and Private Automation Hub

Configuring Supported Sources

In the Ansible Automation Platform 2.7 architecture, Centralized Content Discovery acts as a unified catalog, aggregating automation assets from disparate locations into a single, searchable interface. By configuring external sources, administrators ensure that developers and operators have immediate access to validated collections and roles without manual intervention.

Architectural Overview

The platform allows for the orchestration of multiple content repositories, including:

- **Git Repositories (GitHub/GitLab):** Used for custom automation content, playbooks, and roles managed under version control.
- **Private Automation Hub:** The enterprise source for certified and validated Ansible Content Collections and execution environment definitions.

Administrators maintain granular control over these integrations by defining exactly which repositories are discoverable, applying corporate filtering policies to ensure compliance and security.

Configuring Content Sources

To ensure the platform has visibility into your automation ecosystem, you must register each source and configure its synchronization parameters.

Adding a GitHub/GitLab Source

When adding a Git-based repository, the platform authenticates against your organization to index available assets.

1. Navigate to **Content Management** > **Sources**.
2. Click **Add**.
3. Select the **Source Type**: choose **GitHub** or **GitLab** as required.
4. Provide the Organization or Repository URL.
5. Enter your authentication credentials (Personal Access Token or OAuth2 token).
6. Click **Save**.

Adding a Private Automation Hub Source

Integrating a Private Automation Hub allows for the automatic ingestion of validated collections.

1. On the **Sources** configuration page, click **Add**.
2. Select source type: **Private Automation Hub**.

3. Enter the **Automation Hub URL** (e.g., `https://<your-hub-url>/api/galaxy/`).
4. Provide the necessary authentication (Token or Username/Password).
5. Click **Save**.

Hands-on Lab: Configuring Multi-Source Synchronization

This activity demonstrates how to aggregate content from both a GitHub organization and your enterprise Private Automation Hub.

1. **Navigate to Source Configuration** Log in to the Automation Portal as an administrator and go to the **Git Repositories / Sources** configuration page.
2. **Add Private Automation Hub**
   - Click **Add**.
   - Select **Private Automation Hub**.
   - Enter the provided API URL and credentials.
   - Click **Save**. Verify that the hub appears in your source list.
3. **Trigger a Manual Sync** To populate the catalog immediately, perform a manual synchronization:

   Click the **Sync now** button located at the top right of the Sources dashboard.

   In the selection dialog, select both:

   ☑ GitHub Organization: `<your-org>`

   ☑ Private Automation Hub

   Click **Start sync**.
4. **Verify Completion**
   - Monitor the status indicators. The labels will transition from **"Syncing..."** to **"Synced"** or **"Completed"**.
   - Once synced, navigate to the unified catalog to browse the aggregated collections.

Administrators can apply custom filtering to these sources to align with corporate boundaries, ensuring that only approved content enters the developer workflow. Continuous synchronization can be scheduled to ensure the catalog remains updated as new versions of collections are published.

=== Implementing continuous synchronization and scheduling

Implementing Continuous Synchronization and Scheduling

The efficiency of an automation ecosystem relies on the freshness of its content. In Ansible Automation Platform 2.7, the Centralized Content Discovery module provides robust mechanisms to ensure that automation assets—such as Ansible Content Collections—remain up-to-date across the enterprise.

Overview of Continuous Synchronization

Continuous synchronization allows administrators to aggregate content from multiple sources, including GitHub, GitLab, and Private Automation Hub, into a unified catalog. By automating this process, organizations eliminate the manual overhead of updating repositories and ensure that automation engineers always have access to the latest validated modules and roles.

Admins maintain complete administrative control over the timing, frequency, and source scope, ensuring that synchronization processes do not impact network bandwidth or server performance during peak hours.

Synchronization Modes

The platform supports two primary modes for content updates:

- **On-Demand Sync:** Administrators can trigger an immediate synchronization process by selecting "Sync now" within the Automation Portal UI. This is ideal for urgent hotfixes or immediate deployment of newly published content.
- **Scheduled Sync:** Admins can configure recurring synchronization tasks based on specific time intervals or cron-style schedules. This ensures the catalog remains current without manual intervention.

Configuring Synchronization Schedules

To implement an automated synchronization schedule, follow these administrative steps within the Centralized Content Discovery interface:

1. **Navigate to Sources:** Access the **Content Sources** dashboard in the Automation Portal.
2. **Define Source Scope:** Select the target source (e.g., a specific GitHub organization or Private Automation Hub instance).
3. **Configure Frequency:** Under the **Sync Settings** tab, toggle the "Enable Automated Sync" option.
4. **Define Interval:** Input the desired frequency (e.g., daily at 02:00 UTC or every 6 hours).
5. **Save and Apply:** Upon saving, the scheduler registers the task.

The UI provides real-time visibility into the status of these sync operations. Admins can view the success, failure, or "in-progress" state for each individual source, along with timestamps of the last successful pull.

Hands-on Lab: Configuring and Monitoring Sync

In this exercise, you will configure an automated sync for a repository and verify the process via the dashboard.

Step 1: Configure an Automated Source Sync

1. Log in to the Automation Portal as an administrator.

2. Navigate to **Content Discovery** > **Sources**.
3. Click **Add Source** and provide the credentials for a Private Automation Hub instance.
4. Locate the **Synchronization Schedule** section.
5. Set the schedule to `0 0 * * *` (Daily at Midnight).
6. Click **Save**.

Step 2: Triggering an On-Demand Sync

1. From the **Sources** list, locate the newly created source.
2. Click the **Actions** menu (represented by three dots).
3. Select **Sync Now**.
4. Observe the status indicator; it should transition from "Pending" to "Syncing" and finally to "Success."

Step 3: Verifying Content Catalog

1. Navigate to the **Catalog** tab.
2. Search for a specific collection known to exist in the source you just synchronized.
3. Confirm that the version displayed reflects the latest version retrieved from the remote source.

Troubleshooting Synchronization

If a synchronization task fails, consult the following:

- **Credential Validation:** Ensure that the API tokens or service account credentials provided for the source have the necessary read permissions.
- **RBAC Permissions:** Verify that your current user role includes the `sync_content` permission scope.
- **Log Inspection:** Check the platform logs via the OCP CLI (`oc logs <automation-portal-pod>`) to identify network timeouts or 401/403 authentication errors from the upstream source.

=== Workflow for browse-before-build and tracing existing automation assets

Workflow for browse-before-build and tracing existing automation assets

In large-scale enterprise environments, automation content often suffers from fragmentation. Collections reside in diverse locations—private automation hubs, public Git repositories, or siloed within local team projects. This "silo effect" leads to redundant development and makes it difficult for platform engineers to maintain security standards.

The **Browse-Before-Build** workflow solves this by providing a unified catalog that acts as the single source of truth for all automation assets within the Ansible Automation Platform (AAP) 2.7.

The Browse-Before-Build Philosophy

The core principle of this workflow is to maximize reuse. By shifting the developer mindset from "write new" to "discover existing," organizations can ensure that:

- **Redundancy is minimized:** Teams do not waste time recreating modules or roles that already exist.
- **Consistency is maintained:** Official, vetted collections are used across the enterprise.
- **Visibility is maximized:** Stakeholders can trace the lineage and activity of any automation asset.

Tracing and Discovering Assets

The Unified Catalog architecture aggregates metadata from multiple sources, allowing users to trace assets effectively through the following lifecycle:

1. **Catalog Discovery:** Developers search the unified catalog using keywords, tags, or source metadata to identify existing Ansible Content Collections or roles.
2. **Asset Tracing:** Every entry in the catalog includes metadata that allows you to:
   - **Trace Playbooks:** View which playbooks utilize the collection.
   - **Track CI Activity:** See the last successful build status and recent pull requests associated with the collection.
   - **Access Source Links:** Navigate directly to the original repository (e.g., GitHub, GitLab) to inspect the underlying source code or documentation.
3. **Selection for EE Composition:** Once an asset is identified, it is selected directly from the catalog interface when configuring an Execution Environment (EE) in the Visual Builder.

Hands-on Lab: Exploring the Unified Catalog

This activity demonstrates how to locate and verify an existing automation asset before attempting to build a custom environment.

Ensure your Automation Portal has been configured with at least one external source (GitHub or Private Automation Hub) before starting this exercise.

Step 1: Searching for Collections

1. Navigate to the **Automation Portal** and select **Content Discovery** from the sidebar.
2. In the search bar, enter a keyword relevant to your target infrastructure (e.g., `networking`, `cloud`, or `security`).
3. Filter the results by **Source** (e.g., `Private Automation Hub`) to narrow down the scope to vetted enterprise assets.

Step 2: Evaluating and Tracing an Asset

1. Click on a Collection name from the search results to open its **Detail View**.
2. Examine the **Lineage** tab to view the source link. Click the link to verify the original repository.
3. Check the **Usage** tab; here, you can see if this collection is currently being consumed by other EE definitions or local playbooks within the organization.

Step 3: Integrating into the Visual Builder

1. Once you have identified a collection suitable for your project, click the **Add to EE Draft** button.
2. Navigate to the **Visual Execution Environment Builder**.
3. Observe that the selected collection is now staged in your current working definition, bypassing the need for manual dependency management or URL input.

Troubleshooting Tips

- **Missing Content:** If an asset is not appearing, verify that the **Continuous Synchronization** schedule is active in the Portal Settings under "Content Sources."
- **Authentication Issues:** Ensure that the service accounts used for GitHub or GitLab integration have the necessary read-only permissions to scan the repositories.
- **Stale Data:** If you see an asset but the CI activity status seems outdated, trigger a manual "Sync Now" on the configured content source.

== Appendix

FIXME: Content for Appendix...

Resources

Download Course PDF